

Test Plan

Test Plan ID

CS1D-MLB-TEST-PLAN-001

Purpose

This document defines the basic testing approach for the CS 1D baseball stadium project. It focuses on checking completed user-facing features from the user's point of view and recording whether the expected behavior works.

Scope

The current scope covers completed user-facing behavior and basic admin login behavior. Testing is focused on what a user can see and do in the application.

Test Environment

- Application: CS 1D Baseball Stadium project
- Test type: Manual GUI testing from the user's perspective
- Data used: MLB stadium and team data included with the project
- Tester focus: Confirm that visible features work as expected

Features Tested From User Perspective

- Browsing baseball teams and stadiums in the GUI.
- Sorting visible team/stadium information.
- Filtering the stadium list by league.
- Filtering or viewing stadiums by roof type when available.
- Opening a team/stadium detail window and checking that the displayed fields are understandable.
- Admin login behavior for accepted and rejected login attempts.
- Basic navigation between the main page, browse page, detail page, and admin login/dashboard pages.

Features Not Tested From User Perspective

- Internal backend data structure implementation. The user only sees the final displayed data, not how it is stored.
- Database import and setup internals. The user only sees whether the application has usable data.
- Full admin maintenance workflows, including adding, deleting, and updating stadiums or souvenirs.

Overall Test Strategy

Testing is mostly manual black-box GUI testing. The tester uses the program like a normal user, clicks through the available pages, selects controls, enters simple input, and checks the visible result.

Entry Criteria

- The application builds successfully.
- Required project data is available.
- The feature being tested has been implemented.
- The tester can access the needed GUI pages.
- Any required login or test data is available.

Exit Criteria

- The main expected user behavior passes.
- Any failed test is documented.
- Blocking issues are fixed or recorded.
- The completed feature can be reviewed by the team and product owner.

Roles and Responsibilities

- Developers: Implement features, perform initial checks, and fix bugs found during testing.
- Product owner: Runs the manual tests, compares actual results with expected results, and decides whether the feature meets the requirement.
- Scrum master: Tracks testing status, meeting notes, and scrum log updates.

Approval Process

A developer first checks that the feature builds and works at a basic level. The product owner then runs the manual test from the user's point of view.

If the test passes, the product owner can approve the feature as meeting the requirement. If the test fails, the failure is documented, the issue is fixed by the developer, and the feature is retested by the product owner.

Configuration Management

GitHub is used for version control. The **master** branch is treated as the stable merged project branch.

Feature or member branches are used for active work before merging back into **master**. Current or recent branch examples include **Akeli-fileReading**, **Akil-databaseManager**, **logic_shahob**, **erfan-adminpage-addremove-souvenir**, **alireza-tripplanning-planpage**, and **trip_plan_shahob**.

Changes should be committed with meaningful messages. Testing should be done on the current working branch before merge, and important completed work should be checked again after it is merged into the project branch.

Test Deliverables

The project deliverables follow the problem definition:

- Test Plan
- Agile stories
- Retrospective reports
- Coding standards and team rules
- Scrum log

This file is the Test Plan portion of those deliverables.

Glossary

- Black-box testing: Testing the feature from the user's point of view without using the source code to design the test.
- White-box testing: Testing based on code-level knowledge, such as unit tests, branches, paths, or coverage.
- GUI: Graphical user interface.
- Story: A project requirement written from the user's point of view.
- Expected result: What should happen if the feature works correctly.
- Actual result: What actually happened during testing.